WEEK 1 · DAY 1 · OVERVIEW

Missing Data, Gaps, and Corrections

From your first line of code to your own AI agents.
Every lesson is a fixed income problem.



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 11 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

Every model in this course reads one daily file: the U.S. Treasury's par yield curve, 14 tenors from 2015 to 2026.

A desk meets four kinds of trouble in such a file: market holidays, a feed outage, wrong prints, and series that start late.

The published file holds only the first and the last, so we plant the rest in a copy, from a seeded generator, and keep the true values aside.

We find, correct and fill each problem as if the copy had just arrived, and check every repair against the truth.

Today fits no model, so it works on the whole file.

OBJECTIVES

Tell four kinds of missing apart, count each, and check the dated rows against a market calendar.

Find wrong values by their shape and by a second source, and correct them with a corrections table and a change log.

Find runs of blank cells, fill them with forward fill, backfill and interpolation, score each fill against held-back values, and cap how far a fill reaches.

ABOUT 50 MIN WITH EXERCISES / 11 SECTIONS / 4 EXERCISES

Your Route Through Day 1

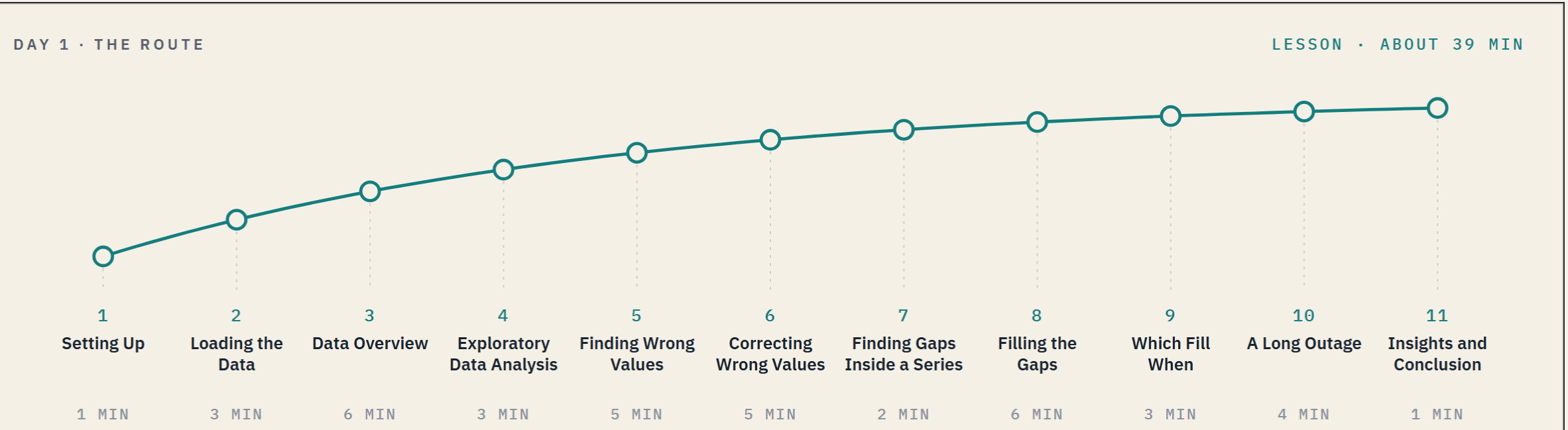


FIGURE 1 Eleven sections, in order, with about how long each takes.

Time. About 39 minutes for the lesson, then about 12 for the four exercises.

Fix the seed and load the libraries.

Setting Up

THE SETUP

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
```

THE LIBRARIES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from ipywidgets import interact
from pandas.tseries.holiday import USFederalHolidayCalendar, GoodFriday
sns.set_theme(style="whitegrid")
sns.set_palette("colorblind")
plt.rcParams["figure.figsize"] = (10, 5)
```

Loading the Data

READING THE TREASURY FILE

```
treasury = pd.read_csv(DATA + "treasury_par_yields.csv", parse_dates=["date"], index_col="date")
print(treasury.shape, "from", treasury.index[0].date(), "to", treasury.index[-1].date())
```

```
(2940, 14) from 2015-01-02 to 2026-10-02
```

THE FILE'S COLUMNS

- **`date`**: the market date, the index
- **`1 Mo`** to **`30 Yr`**: 14 par yields, in percent, shortest tenor first
- **`planted`**: a copy of the file with the planted problems
- **`backup`**: a second source, the desk's own end-of-day sheet for the 2 Yr and the 10 Yr from 2021

THE OUTAGE

```
rng = np.random.default_rng(SEED)
planted = treasury.copy()
outage_start = rng.integers(1_100, 1_400)
outage_dates = planted.index[outage_start:outage_start + 20]
planted.loc[outage_dates] = np.nan
print("outage:", outage_dates[0].date(), "to", outage_dates[-1].date())
```

```
outage: 2019-07-03 to 2019-07-31
```

We work on `planted` as if it had just arrived, and read `treasury` only to score a repair.

Data Overview

date	5 Yr	10 Yr	2 Mo
2018-10-12	3.00	3.15	NaN
2018-10-15	3.01	3.16	NaN
2018-10-16	3.02	3.16	2.22
2019-07-02	1.75	1.98	2.17
2019-07-03	NaN	NaN	NaN
2019-07-05	NaN	NaN	NaN
2022-10-20	4.45	42.40	3.83

not started yet
NaN in every row before the series' first value

blank cells
the row is there, its values are NaN

an absent row
2019-07-04 is not in the index: no row at all

a wrong value
a number, the wrong one: 42.40 where the truth is 4.24

FIGURE 2 Seven rows of the copy: a 2 Mo not yet started, a blank outage row, an absent 2019-07-04 and a 42.40 print.

THE SUMMARY

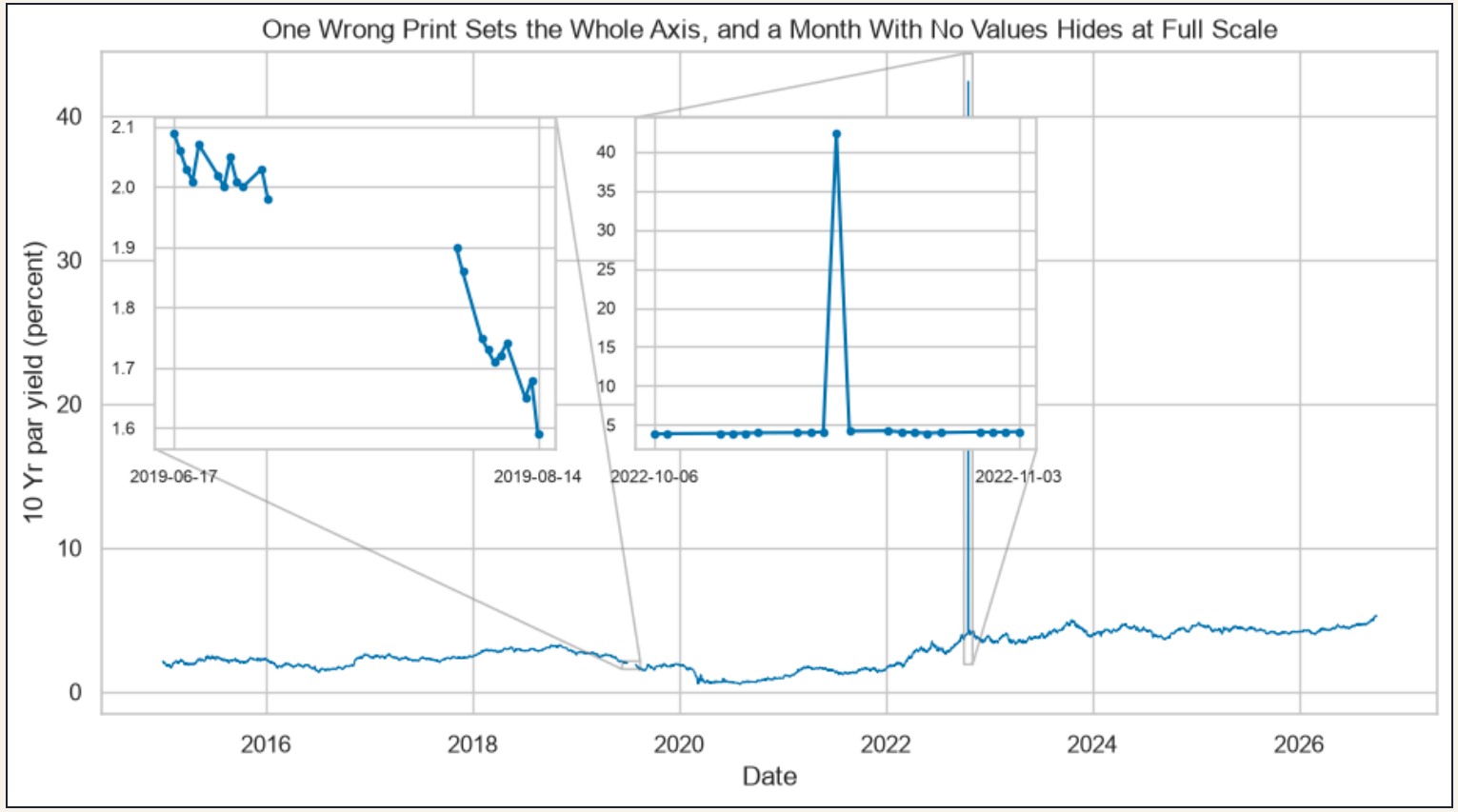
	2 YR	10 YR	30 YR
count	2915.00	2915.00	2915.00
mean	2.33	2.78	3.22
std	2.91	1.40	1.07
min	0.09	0.52	0.99
25%	0.74	1.81	2.43
50%	2.12	2.51	3.01

WEEKDAYS WITH NO ROW

```
3066 weekdays, 2940 rows, 126 weekdays with no row
```

Absent rows are the market's calendar, so they stay absent; the blanks and the impossible maximums are ours to explain.

Exploratory Data Analysis



TWO CLOSE-UPS IN ONE CHART

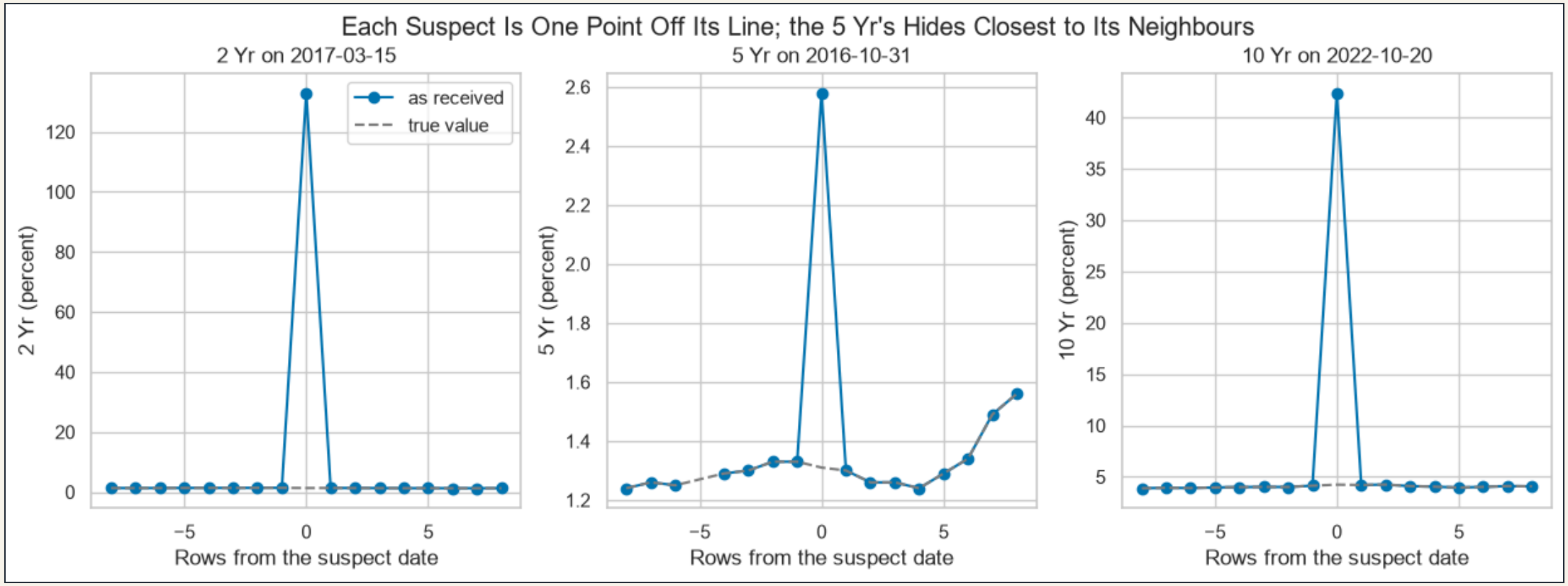
At full scale the 10 Yr is a flat line near the bottom: one print of 42.40 sets the whole axis.

The October 2022 close-up shows it, one point far above its neighbours.

The July 2019 close-up shows the hole, easy to miss at full scale.

Two kinds of trouble need two hunts: numbers that are wrong, and cells that are blank. Wrong numbers come first, because a correction can leave a blank.

Finding Wrong Values



Three wrong values found two ways. Next we correct them so that each change leaves a record.

Correcting Wrong Values

A CORRECTIONS TABLE

	DATE	COLUMN	VALUE
0	2016-10-31	5 Yr	NaN
1	2017-03-15	2 Yr	1.33
2	2022-10-20	10 Yr	4.24

ONE CELL BY HAND

```
by_hand = planted.copy()
by_hand.loc[wrong_10y, "10 Yr"] = backup.loc[wrong_10y, "10 Yr"]
print(round(planted.loc[wrong_10y, "10 Yr"], 2), "became", by_hand.loc[wrong_10y, "10 Yr"])
```

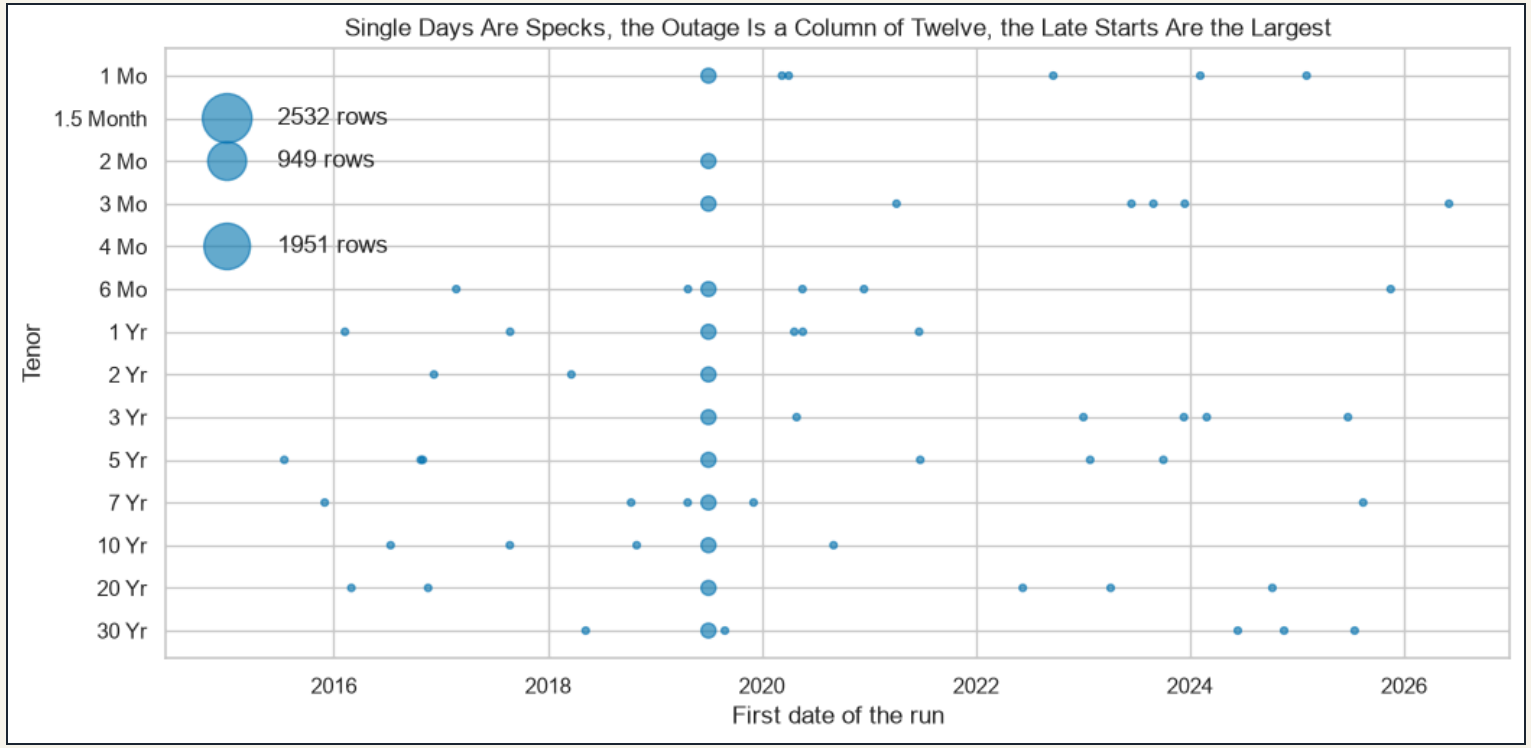
```
42.4 became 4.24
```

APPLYING THE TABLE

```
updated = planted.copy()
updated.update(corrections.pivot(index="date", columns="column", values="value"))
for date, column in zip(corrections["date"], corrections["column"]):
    print(column, date.date(), round(planted.loc[date, column], 2), "became", round(updated.loc[date, column], 2))
```

It works, and nothing records what changed, from what, or why.

Finding Gaps Inside a Series



HOW LONG EACH RUN LASTS

```
rows
1       52
20      12
2532     1
949      1
1951     1
Name: count, dtype: int64
```

THE CELLS TO FILL

```
52 single-day cells, 240 outage cells
```

One blank day and a 20-day outage are different problems, so every fill is scored on both.

Filling the Gaps

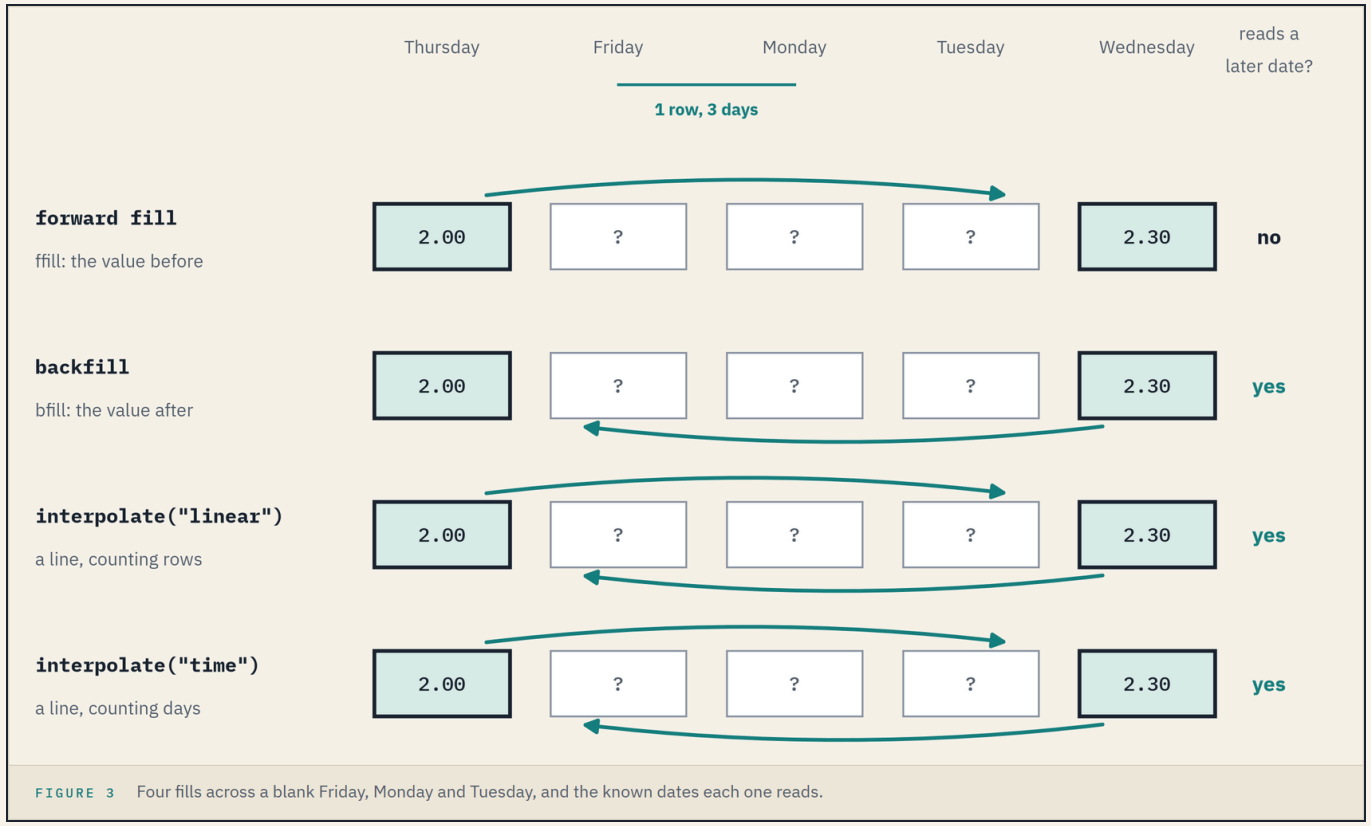


FIGURE 3 Four fills across a blank Friday, Monday and Tuesday, and the known dates each one reads.

FOUR FILLS, CELLS LEFT BLANK

```
ffill 5432 cells still blank
bfill 0 cells still blank
linear 5432 cells still blank
time 5432 cells still blank
```

WHERE THE 2 MO STARTS

```
bfill 949 values before 2018-10-16
limit_direction="both" 949 values before 2018-10-16
limit_area="inside" 0 values before 2018-10-16
```

The backfill and the fill in both directions put 949 values before the 2 Mo started, flat at its first value.

Which Fill When

THE CUT TEST

Course 3's cut test asks whether a value for a date changes when later rows arrive: run the same steps on the whole file and on the file cut at a date, and compare the rows up to and including the cut.

We cut on the outage's tenth date.

THE QUESTION

Q. Up to the cut the outage holds 120 blank cells, 10 dates in 12 tenors. For how many of them does the filled value change between the whole file and the cut file: with the forward fill, and with the time interpolation? Write down two numbers.

WHICH FILL FOR WHAT

Forward fill for a quote you could have traded on that date: the last value published.

Interpolation for a point on a curve, or a description of history, where both ends are known.

EACH FILL ON THE WHOLE FILE AND ON THE CUT FILE

```
cut = outage_dates[9]
cut_fills = {"ffill": repaired.loc[:cut].ffill(), "time": repaired.loc[:cut].interpolate(method="time")}
for name, cut_fill in cut_fills.items():
    changed = (fills[name].loc[:cut] != cut_fill) & outage_cells.loc[:cut]
    print(name, changed.sum().sum(), "of", outage_cells.loc[:cut].sum().sum(), "outage cells up to", cut.date(), "changed")
```

A Long Outage

STOPPING THE FILL AFTER FIVE BLANKS

```
limited = repaired.ffill(limit=5)
still_blank = limited["10 Yr"].isna()
print("10 Yr dates still blank:", still_blank.sum(), "from", limited.index[still_blank][0].date(), "to", limited.index[still_blank][-1].date())
```

```
10 Yr dates still blank: 15 from 2019-07-11 to 2019-07-31
```

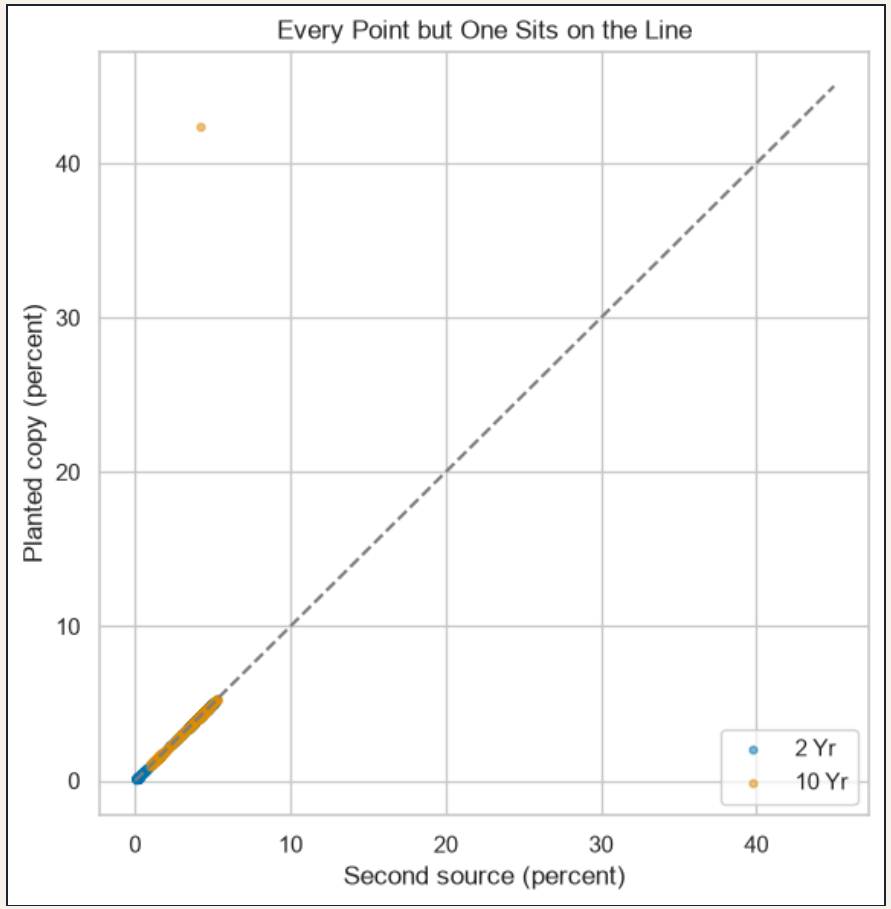
KEEPING ONLY THE DATES WITH A VALUE

```
kept = limited["10 Yr"].dropna()
print(len(kept), "of", len(limited), "dates kept;", len(limited) - len(kept), "left out")
```

```
2925 of 2940 dates kept; 15 left out
```

A capped fill admits what it does not know, and the score says how many dates it left out.

Insights and Conclusion



TWO SOURCES SIDE BY SIDE

```
both = planted.loc[backup.index, backup.columns]
gap_bp = (both - backup).abs() * 100
gap_bp.where(gap_bp > 0.5).stack().dropna()
```

WHERE THEY DISAGREE

```
2874 cells in both, 1 apart by more than half a bp
date
2022-10-20  10 Yr    3816.0
dtype: float64
```

WHAT THE SHEET CAUGHT

One disagreement in 2,874 cells: the 10 Yr on 2022-10-20, 3,816 bp apart.

Day 2 builds changes and lags on the published file, which has no blank inside any tenor's span.

You Can Now

- Tell an absent row, a blank cell, a series not started yet and a wrong value apart, and count each.
- Check dated rows against the market calendar.
- Find wrong values by their shape and by a second source, and correct them with a corrections table and a change log.
- Find runs of blank cells, fill them four ways, and score each fill against held-back values.
- Test a fill with the cut test, and cap it with `limit=`.

NEXT
WEEK 1 · DAY 2

Frequency, Lags, and the Naive Baselines

Day 2 sets the cut, builds lags and changes, and meets the rules every model must beat.